

AI 赛事陪伴 说明稿

一、用户使用场景

我是小林，看了八年 LPL 的死忠，工作繁忙，但熬夜追比赛雷打不动。

- 凌晨一点半，我开始看我期待已久的决赛。凌晨一点半,我一个人守着等了一个月的决赛。决胜局,对面五个全压去抢大龙,我方一个人绕后——偷家!水晶直接爆了,绝杀翻盘!我从沙发上弹起来:"卧槽你看这个偷塔,太绝了!"喊完才反应过来,屋里就我一个,没人理。我抓起手机给女朋友发消息,她回我:"偷塔是什么意思?"
- 局势开始无聊，一直在磨,无聊得我手指已经有点想滑到退出键，要是有人和我一起吐槽吐槽就好了。
- 赢了!想发条朋友圈,憋半天只憋出"YYDS",配图文案全网撞款。算了,删了。

一句收尾(产品登场) 小林要的不多——一个 2am 也在线、陪他共振、懂他黑话的搭子。

二、用户群体刻画

用户画像

- 一句话：熬夜独自看比赛、情绪起伏巨大、表达欲极强却没处宣泄、看完想发社交平台记录却懒得找素材写文案的高频电竞观众。
- 核心标签:**无聊观赛者 · 高频核心电竞迷 · 强阵营认同 · 情绪上头型 · 社交活跃分子

用户痛点

“没人即时回应你” “没人帮你写发社交平台记录的素材和文案”

虎扑等直播平台和电竞社区聚焦赛中公共讨论，偏向大锅饭式的群体狂欢，个体情绪和表达易被淹没。

通用AI平台助手缺乏对电竞狂热粉需求的理解，不感知比赛，整体表达缺乏玩梗、黑化和情绪的承接和带动；赛后生产文案和海报需要用户主动搜罗素材制作，带来用户成本。

电竞观赛是一个完整的闭环，赛前期待、赛中共振、赛后表达。

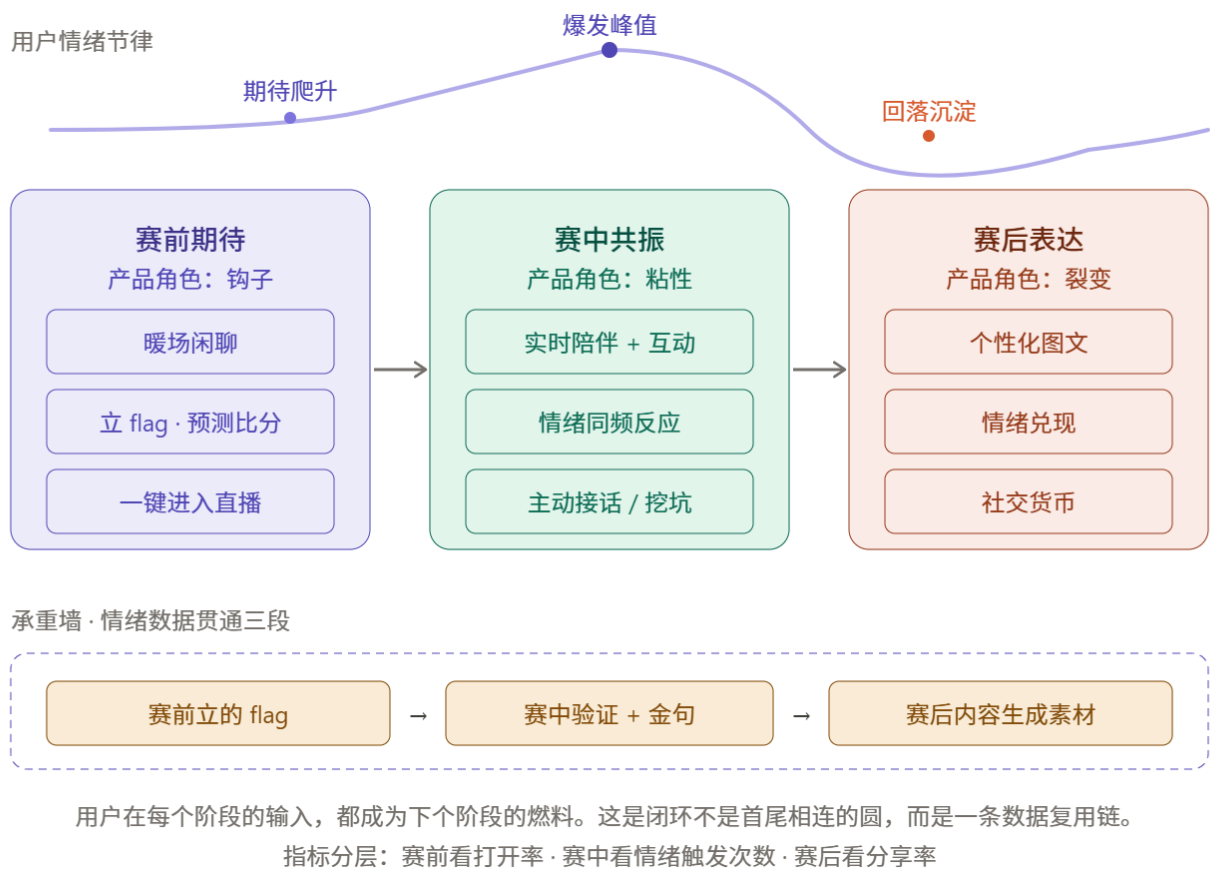
搭子陪伴

致力于全程看见你最定制化的需求！赛前推荐你关注的赛事，和你一起期待；赛中化身兄弟，和你一起玩梗说黑话，蚰蚰对方战队偷塔；赛后化身助手，帮你记录下比赛的高光时刻，生产素材和文案。

三、解决方案拆解

3.1 用户产品交互体验：

用户全程交互体验流畅且情绪完整：赛前快速点燃观赛期待，轻松进入观赛状态；赛中AI 实时陪伴、同频共情，强情绪瞬间触发音效与视觉互动，彻底告别孤独观赛，实现深度情绪共振；赛后一键生成专属个性化图文与文案，把赛中情绪直接兑现为可分享的社交货币，完整闭环 “赛前期待 — 赛中共振 — 赛后表达”，全程懂你、陪你、帮你表达。



3.2 产品支持和策略架构

整个基于用户拆解产品模型的思路是：从why思考到how再是what

底层锚定（WHY）：以差异化价值为地基，四阶段分别锚定「朋友感 / 仪式感 / 情绪同频 / 个人印记」，拒绝无意义功能堆砌，确保产品不可替代。

中层支撑（HOW）：

- 数据流：静态偏好 → 时间触发 → 实时上下文 → 结构化日志，形成完整数据闭环。
- 技术栈：用户系统 / 推送机制 / LLM + 规则引擎 / AIGC + 日志系统，重投入赛中核心场景，其余轻量化。

上层落地（WHAT）：

- 系统阶段：建档 → 推送 → 陪伴 → 创作。

- 核心能力：画像 → 提醒 → 对话 → 生成。
- 完整覆盖「注册 - 赛前 - 赛中 - 赛后」用户旅程。

用户旅程时间轴



策略阅读顺序：自下而上
先定每列的差异化（最底层）→ 倒推数据流和技术 → 最后落到系统能力和用户旅程。差异化是产品的“地基”，不是装饰。

资源分配读图：赛中（深色）= 重投入；其余列 = 轻量化保证体验完整即可
如果某列差异化填不出来，那一列就不该做——这是保持产品刀刃锋利的关键

3.2.1 赛前播报

3.2.1.1 用户痛点

"看球这件事是孤独的，我想有人陪" —— 但现实中：同事不看球、爱人要睡觉、朋友支持的队和你对立。赛前30分钟，你一个人打开直播，没有仪式感，没有期待感。

3.2.1.2 传统方案缺陷

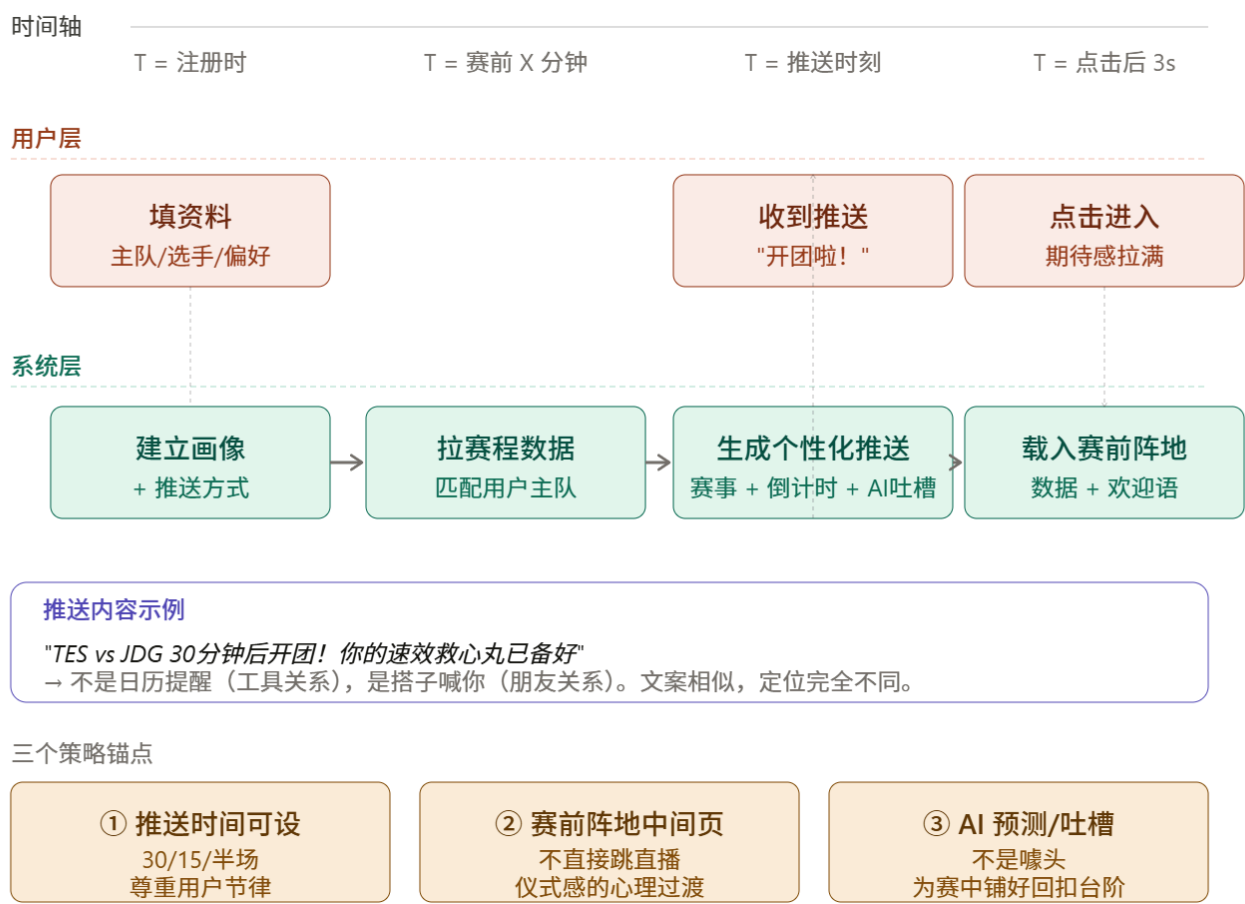
方案	缺陷
虎扑/懂球帝推送	千人一面，只报"XX vs XX即将开始"，不够认识你，传统推送带不来邀约互动的感觉
日历提醒	冷冰冰的系统通知，没有情绪预热

3.2.1.3 我们的解决框架

同样是一条推送，"虎扑：TES vs JDG 即将开始"和"搭子：30分钟后开团，速效救心丸已备好"，文字内容差不多，但用户感知完全不同——前者是工具关系，后者是朋友关系。这个差异不是文案技巧，是**产品身份的策略选择**：你定位自己是冷冰冰的赛事数据库，还是那个 2am 也在线的兄弟。一旦定位选了，整个推送链路的每一步都要服务于这个定位。

策略上有三个关键决策点：

- ① **推送时间可设置（30/15/半场）**——尊重用户的不同节律。死忠粉可能要30分钟提前进入状态，工作党可能只需15分钟，让他选，他就觉得这个搭子"懂规矩"。
- ② **"赛前阵地"这个中间页**——为什么不直接跳直播？因为用户需要一个"准备"的小停顿。这30秒里搭子说一句欢迎语、给你看双方数据、推荐看点，仪式感就立住了。直接跳直播会丢掉这个心理过渡。
- ③ **推送内容里"AI预测/吐槽"不是噱头**——它是一根线，把赛前的预言挂到赛中去验证。"我赌TES上路20分钟崩盘"这种预测立在赛前，赛中真崩了，搭子就有了"看吧我说什么来着"的回扣点。赛前的一句吐槽，是给赛中铺好的台阶。



核心设计：把"系统通知"变成"朋友喊你开黑"，用主队视角建立归属感。

3.2.2 赛中反馈

3.2.2.1 用户痛点

"赛场瞬息万变，精妙战术、争议判罚、绝杀狂喜——却无处可说、无人可分享"。弹幕是弱交互单向输出，无法对个体提供定制化共情回应。

更深层：用户需要的不是"信息"，是情绪共振。赢了要有人一起吹，输了要有人一起骂，平淡时要有人主动暖场。

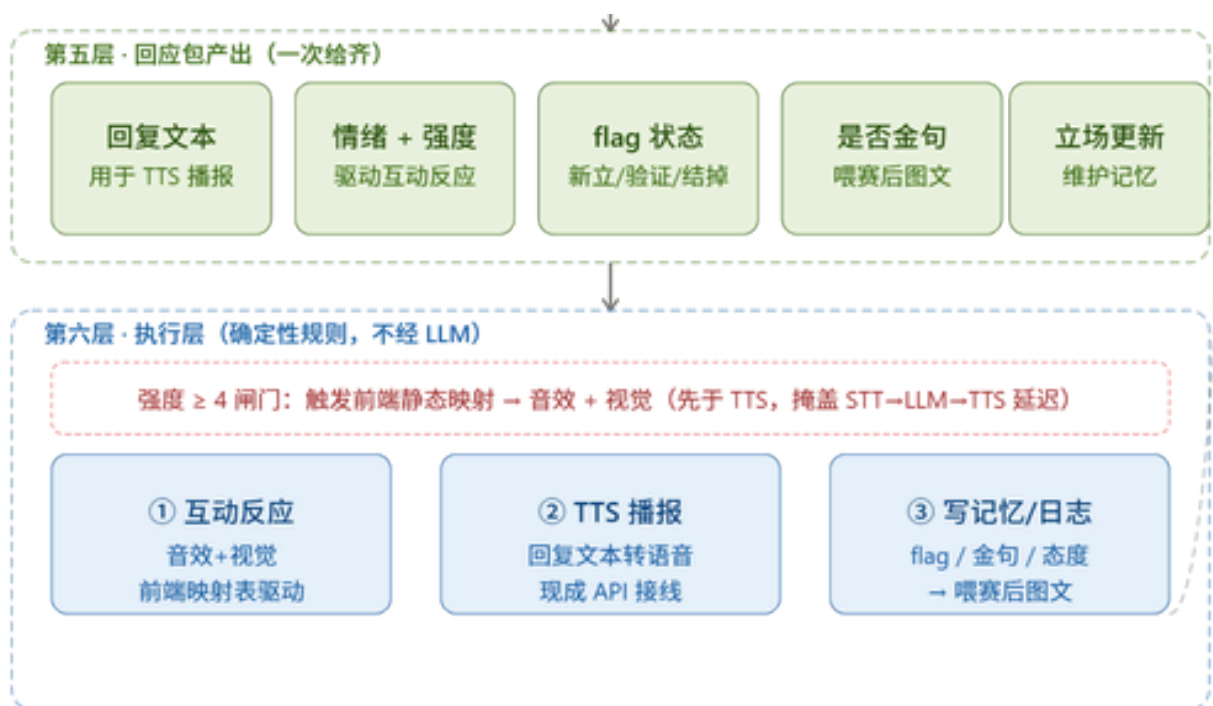
3.2.2.2 传统方案缺陷

方案	缺陷
赛事直播弹幕	大锅饭，信息噪音，不向着你的队，不认识你
纯LLM聊天机器人	中立播报、长篇分析、没有态度、记不住你是谁
虎扑赛后帖	赛后才能发，赛中情绪凉了
语音助手 (Siri/Alexa)	查比分可以，聊情绪不行

3.2.2.3 我们的解决框架

永远站在用户这边，用最短延迟、最懂情绪的方式完成「实时陪伴 + 情绪共鸣 + 记忆沉淀」，解决用户观赛时“情绪无处说、没人懂我立场”的痛点。（具体策略设计机制见PRD）





情绪标签是全链路中枢·LLM 只产出 (情绪, 回复, 日志)·音效/视觉走前端静态映射独立扩展

3.2.3 赛后创作

3.2.3.1 用户痛点

"想把热爱发出去时，又懒得自己憋字&自己做海报"。普通球迷有强烈表达欲，但受限于文字提炼能力和战术理解深度，难以产出高质量"球评"或"心情图文"。

更深层：发一张"主队logo+比分"的模板图，所有主队球迷发出来一模一样——没有"这是我"的质感。

3.2.3.2 传统方案缺陷

方案	缺陷
赛事官方海报	大白化，千人一面，没有个人情绪
自己截图+配文	创作门槛高，赛后情绪凉了才发
AI生成通用文案	"这就是足球"式套话，没有用户原话

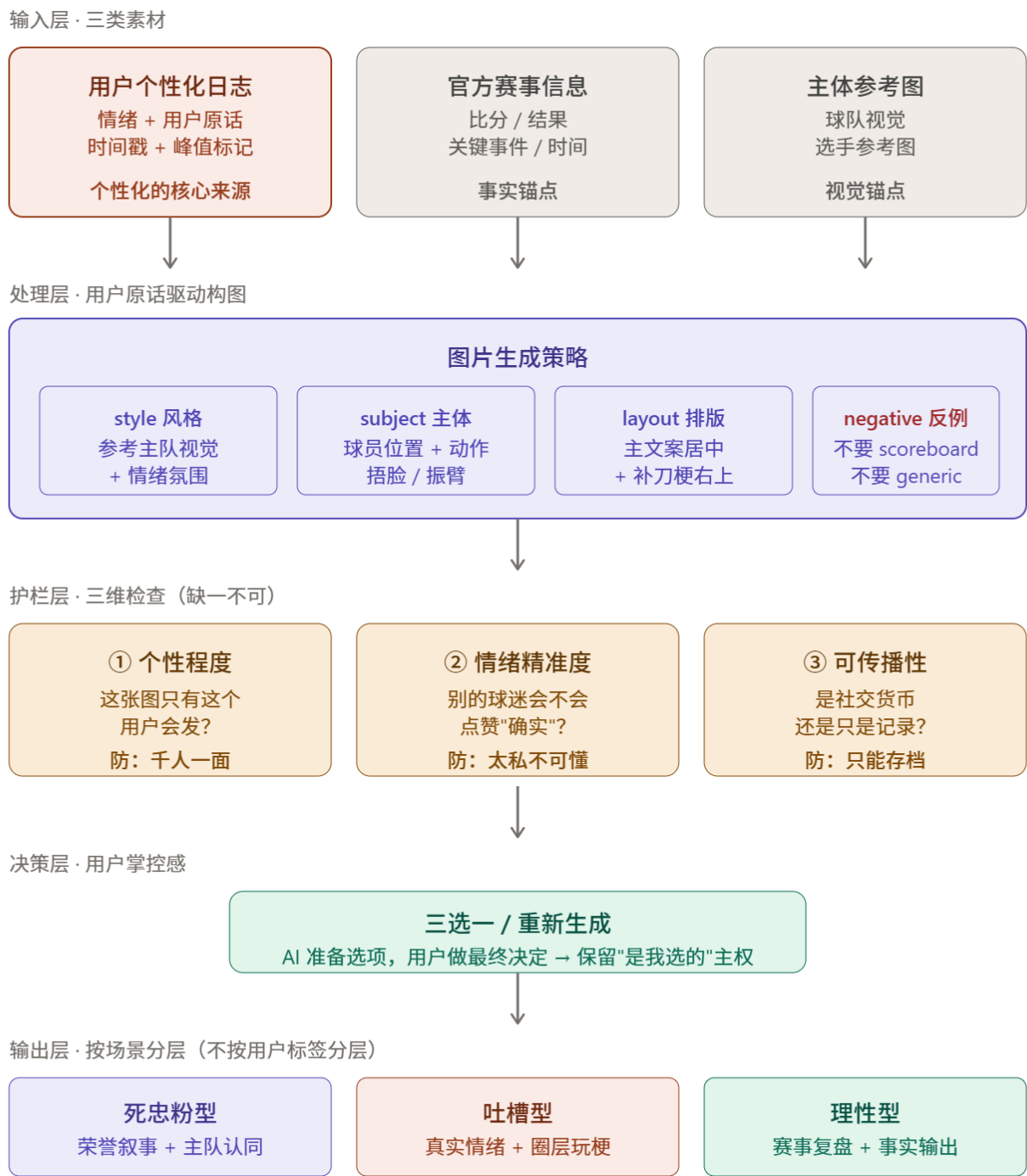
3.2.3.3 我们的解决框架

把"AI替你写"变成"AI放大你"。市面上所有AI生成内容产品都在做前者——给个主题，AI输出一段文字。这种产品的根本问题是：用户看到生成的东西时，会觉得"这是AI的，不是我的"，没有发出去的动力。我们反过来——AI不再写内容，AI只是把用户赛中真实说过的话、真实经历的情绪、真实立过的

flag结构化和放大。最后那张图、那段文字，每一个核心元素都来自用户自己，只是被组织得更好看了。这是个性化的根本，不是配色和字体。

最值得说的策略设计是"三维检查"这一关。 个性度、情绪精准度、可传播性——这三个维度是相互制约的护栏：

- 如果只要个性度高 → 就变成日记，只有用户自己懂，没人愿意转发
- 如果只要情绪精准度高 → 像数据报告，发出去太理性，不像朋友圈
- 如果只要可传播性 → 又变回千人一面的模板图，失去"这是我"的感觉



3.3 底层技术架构

一、顶层判断：客户端为主，服务端为辅

整个技术方案的取舍始于一个约束——**10小时 demo 窗口**。这决定了所有选择都向"客户端承担90%逻辑、服务端只做必要的事"倾斜：没有数据库（localStorage 顶上）、没有真实赛事API（脚本化 Mock层）、没有实时通信协议（HTTP 调用代替 WebSocket）。

这不是技术能力问题，是策略性的"剪枝"——把工程复杂度让给产品体验。这种取舍在 demo 阶段是对的，因为评委看不到数据库长什么样，但能看到 demo 流不流畅、反应快不快、视觉打不打动。

二、三层 + 一个旁路

读懂原架构图的关键是看出"四块"而不是"三层"：

① 客户端 UI · src/routes

6 条文件系统路由是产品 6 个阶段的物理映射：`/welcome` → `/onboarding` → `/chat` → `/pre-match` → `/match` → `/post-match`。每条路由是一个独立的"产品时刻"，承载完全不同的 UI 状态和交互逻辑。`__root.tsx` 提供 Global Layout 和过渡动画，保证跨页面体验连贯。

② 客户端状态 · Zustand + TanStack Query + localStorage

Zustand 是这个产品的"中枢神经"——`esports-buddy state` 里包含用户画像、对话记录、flag 清单、情绪日志。所有 UI 从这里读，所有变更写回这里。`localStorage persist` 让数据跨刷新存活——但这本质上是把 **localStorage 当数据库用**，MVP 合理，生产化时第一个要换。TanStack Query 在 demo 阶段负担很轻，因为大部分数据已经在客户端。

③ 服务端 · Vite SSR + AI API Gateway

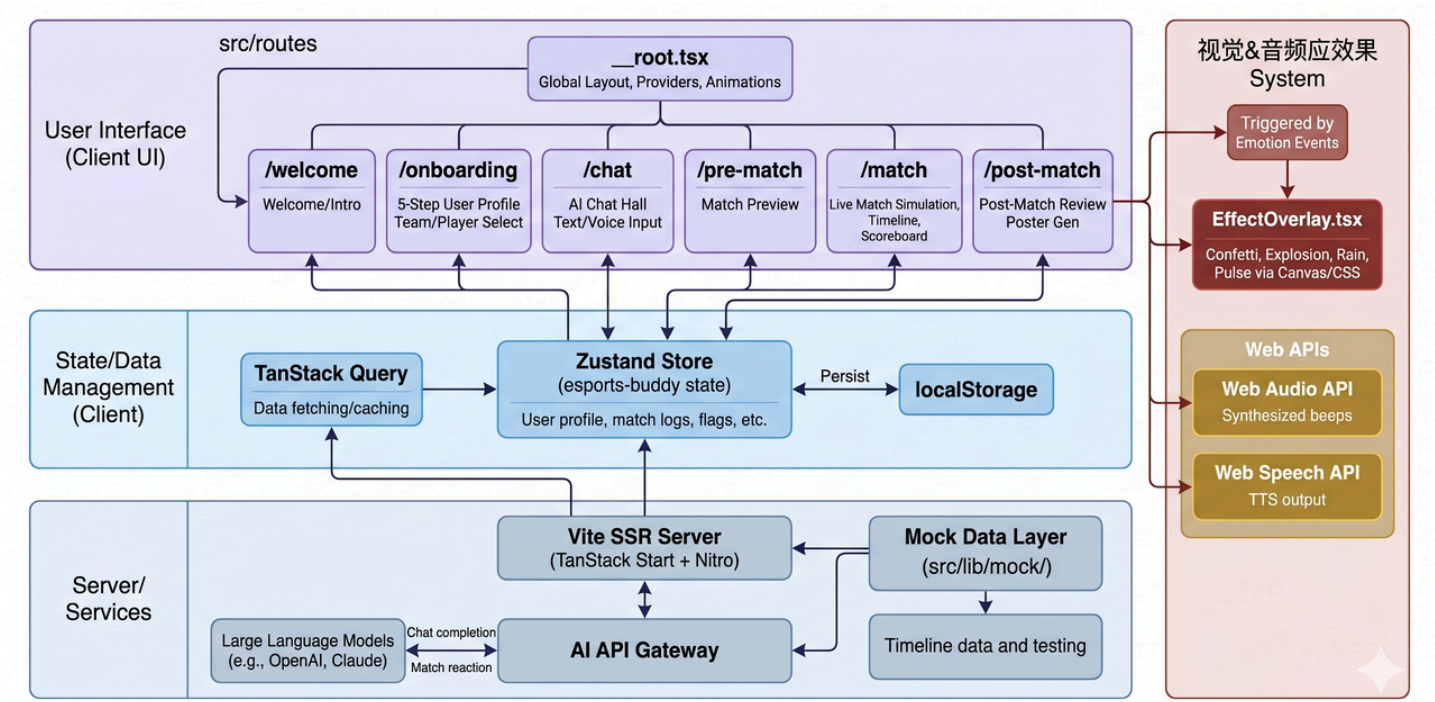
服务端就做两件事：SSR 让首屏快；AI API Gateway 作为 LLM 代理层。Gateway 这个设计**值得单独说**——它把"调用哪家 LLM"的选择从客户端抽离出来，未来切换 OpenAI / Claude / 国产模型时客户端不用改一行代码。这是个看似不起眼但极其重要的抽象。Mock Data Layer 提供脚本化的赛事时间线，是 demo 能稳定演出的关键——比真实 API 还可靠，因为每个高光点都掐着秒来。

④ 旁路 · 视觉&音频效果系统

这是整个架构最值得讲的部分——它**不属于上面任何一层**，是一个独立的旁路。情绪事件从 Zustand 触发 `EffectOverlay`，直接调用 Web Audio API 和 Canvas/CSS 出效果，**完全不经过 LLM**。这个解耦是产品"反应先于语音"的技术基础——下一节展开。

层级	技术选型	用途
框架	React	UI 组件基础
路由	TanStack Router	文件系统路由 + SSR
全栈	TanStack Start	服务端渲染 + API 路由
构建	Vite	开发服务器 + 打包

样式	Tailwind CSS	原子化 CSS
状态	Zustand	全局状态管理
动画	Framer Motion	页面过渡 + 交互动画
查询	TanStack Query	服务端数据获取



四、我们的终极理想态

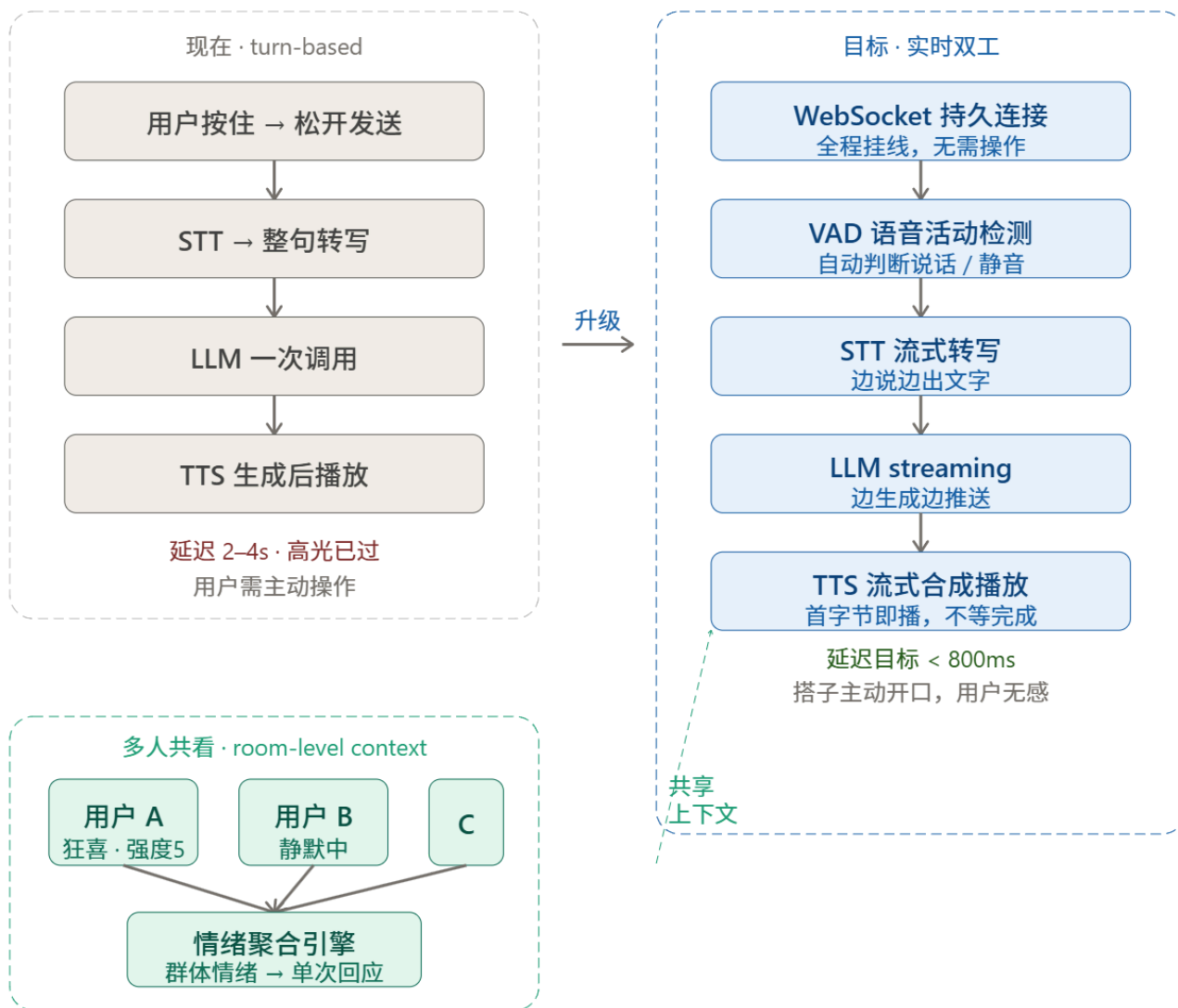
4.1 把陪伴做成更沉浸的理想体验

现在的产品是**turn-based**：用户按住说，搭子回。这个门槛其实挺高的——高光瞬间你根本不想掏手机

理想态是：

先达成单人的沉浸体验：WebSocket持久连接，VAD（语音活动检测）实时监听，用户不用任何操作，说话就有反应。搭子变成真正挂在耳边的存在，比赛激烈时它主动开口，用户沉默时它等着，不打扰。

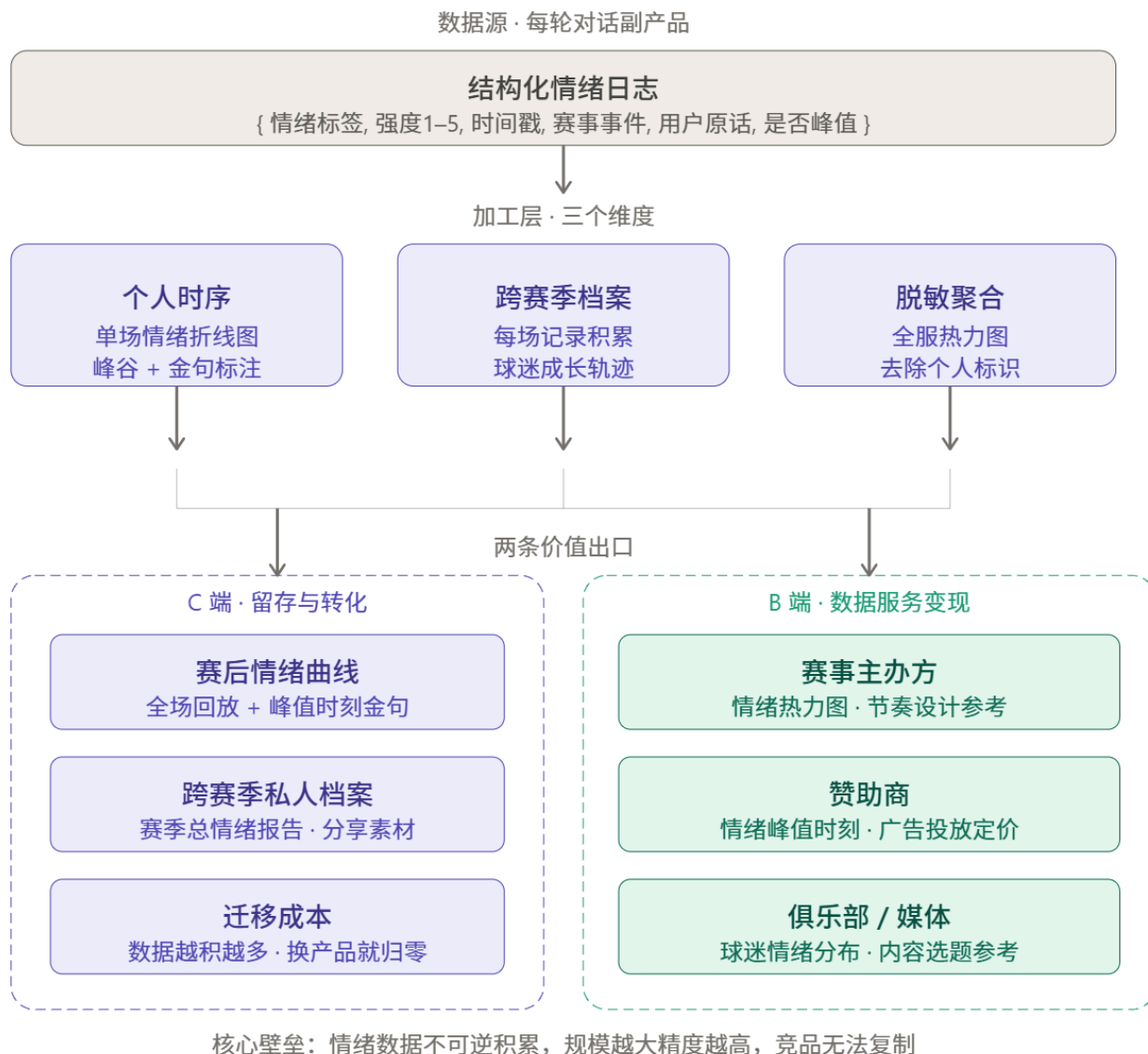
后达成多人共看的沉浸体验：2-4 人挂同一个房间，搭子能感知到"阿哲说了一句话、小林在静默、阿强刚刚发出了狂喜情绪"，把群体情绪聚合后再回应，而不是对每个人单独回复。技术上是一个 room-level context 覆盖在各自的 user-profile 上。



4.2 造一个独属于你的电竞情绪的记忆空间

现在的产品是turn-based: 赛时的情绪曲线赛后看不见，没有单独的电竞热血记忆空间存下每场比赛的情绪

理想态是: 每轮对话都在产出结构化情绪日志：{情绪标签，强度，时间戳，赛事事件，用户原话，是否峰值}。赛后把全场情绪折线图还给用户，峰谷标注金句和对应赛事时刻，建立"私人观赛档案"，电竞情绪的记忆空间保住了一部分用户留存，同时情绪日志做为数据资产反哺后续的商业化变现



4.3 打造产品和生态的护城河

当极致体验和数据壁垒都立起来后，最终平台化才是护城河。

情绪识别模块、回复策略模块、记忆系统、互动反应框架——四个模块各自独立，通过标准API暴露。俱乐部可以把官方吉祥物套进来，用我们的情绪引擎做自己的球迷搭子；内容平台可以把情绪引擎嵌进直播间，和弹幕并存；个人开发者可以针对小众赛事建自己的垂直搭子。

这个时候，我们做的已经不是一个应用，而是"体育情绪陪伴"这件事的基础设施。任何想做这件事的人，都得在我们的引擎上跑。

搭子 OS · 情绪基础设施平台

任何赛事 · 任何场景 · 情绪引擎统一驱动

